IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

David Troletti
2022-04-17

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Methodologies used to analyze data
  - Data was collected through webs craping and the SpaceX API
  - The data was then cleaned up to help provide more useful information
  - The more useful data was the split into training and testing sets to create machine learning predictions
- Summary of all results
  - Data was successfully collected from public sources
  - Features were chosen to best predict success of a launch

Introduction

- Project background and context
  - Rival space company SpaceY wants to use data on SpaceX launches to help determine cost of a launch
  - This information will be used to predict the success of the first stage landing to try and outbid SpaceX on contracts
- Problems you want to find answers
  - What determines a successful landing?
  - How do the features interact?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - Data was collected through the SpaceX API and Wikipedia web scraping
- Perform data wrangling
  - One hot encoding was used on categorical features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

Data Collection

- API
  - Data was retrieved using get requests from the SpaceX API
  - .json and .json_normalize were used to process the data and convert to a pandas data frame
  - Missing values were filled in or removed where appropriate
- Web scraping
  - Beautiful soup was used to scrape data from the Falcon9 Wikipedia page
  - Records were extracted as html tables and converted to pandas data frames

Data Collection – SpaceX API

- The get request was used to retrieve the data and underwent processing and wrangling
- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/1_CollectingData.ipynb

Task 1: Request and parse the SpaceX launch data using the GET request

To make the requested JSON results more consistent, we will use the following static response object for this project:

In [9]:
```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json'
```

We should see that the request was successfull with the 200 status response code

In [10]:
```
response.status_code
```

Out[10]: 200

Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

In [12]:
```
data=pd.json_normalize(response.json())
```

Using the dataframe `data` print the first 5 rows

In [13]:
```
data.head()
```

Data Collection - Scraping

- Data was scraped from the Wikipedia page and was parsed and cleaned using Beautiful Soup
- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/1_CollectingData.ipynb

1. Apply HTTP Get method to request the Falcon 9 rocket launch page

In [4]:
```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
```

In [5]:
```
# use requests.get() method with the provided static_url
# assign the response to a object
html_data = requests.get(static_url)
html_data.status_code
```

Out[5]:
```
200
```

2. Create a BeautifulSoup object from the HTML response

In [6]:
```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(html_data.text, 'html.parser')
```

Print the page title to verify if the `BeautifulSoup` object was created properly

In [7]:
```
# Use soup.title attribute
soup.title
```

Out[7]:
```
<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>
```

3. Extract all column names from the HTML table header

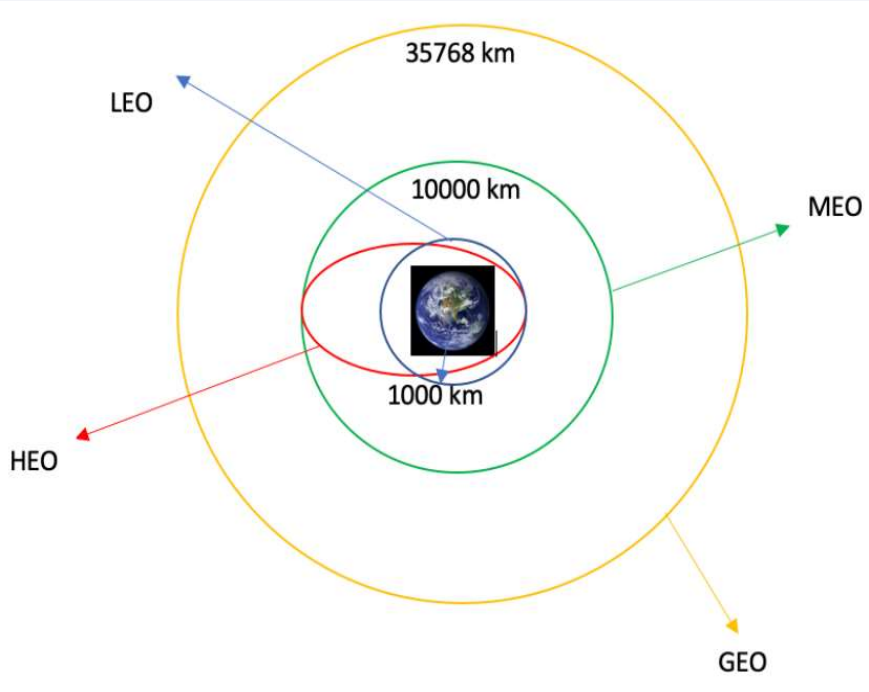
In [10]:
```
column_names = []

# Apply find_all() function with `th` element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the Non-empty column name (`if name is not None and len(name) > 0`) into a list called column_names

element = soup.find_all('th')
for row in range(len(element)):
    try:
        name = extract_column_from_header(element[row])
        if (name is not None and len(name) > 0):
            column_names.append(name)
    except:
        pass
```

4. Create a dataframe by parsing the launch HTML tables
5. Export data to csv

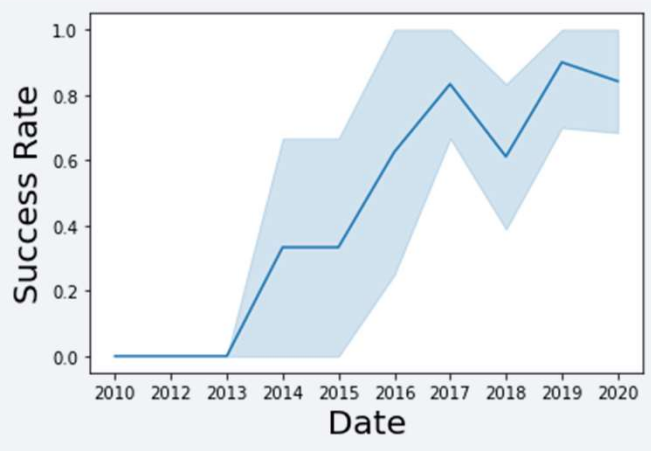
Data Wrangling

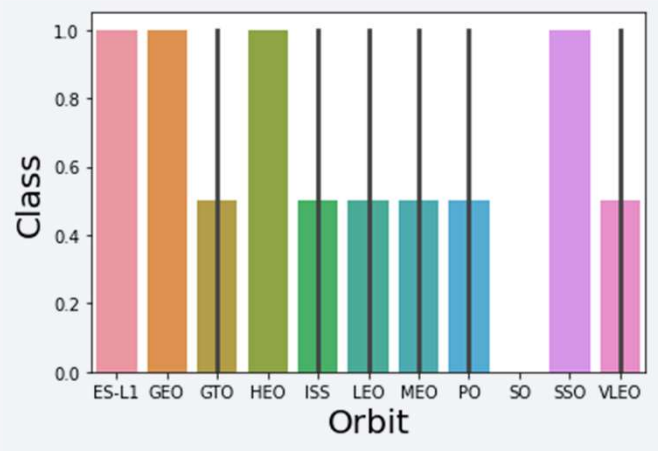


- EDA was performed to determine training labels
- The number of launches at each site and what orbit was reached was calculated
- An outcome landing table was created
- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/2_DataWrangling.ipynb

EDA with Data Visualization

- We visualized the relationship between flight number and launch Site, payload and launch site, success rate of each orbit type, flight number and orbit type, the launch success yearly trend.





- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/4_EDAWithVisualization.ipynb

EDA with SQL

- We applied EDA with SQL to get insight from the data.
- The following queries were made
  - The names of unique launch sites in the space mission.
  - The total payload mass carried by boosters launched by NASA (CRS)
  - The average payload mass carried by booster version F9 v1.1
  - The failed landing outcomes in drone ship, their booster version and launch site names.
- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/3_SQLExploratoryAnalysis.ipynb

Build an Interactive Map with Folium

- Launch sites were added to the folium map and were marked according to distances and successes
- Failures were mapped to a 0 and success were mapped to a 1
- This allowed us to color code and visually see high success rated
- We calculated the distances between a launch site to its proximities. We answered some question for instance:
  - Are launch sites near railways, highways and coastlines.
  - Do launch sites keep certain distance away from cities.
- Notebook Link: https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/5_InteractiveVisualAnalytic.ipynb

Build a Dashboard with Plotly Dash

- An interactive dashboard was made with Plotly Dash
- Pie charts showed total launches based off of selected site
- A scatter plot showed the relationship between outcome and payload mass
- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/6_spacex_dash_app.py

Predictive Analysis (Classification)

- Numpy and pandas were used to split the data into test and training sets
- Different machine learning models were made and tuned different hyperparameters using GridSearchCV
- Accuracy was the main metric for our model improving it using feature engineering and algorithm tuning.
- The link to the notebook is https://github.com/davtro1234/DataScience/blob/main/CapstoneProject/7_MachineLearningPrediction.ipynb

Results

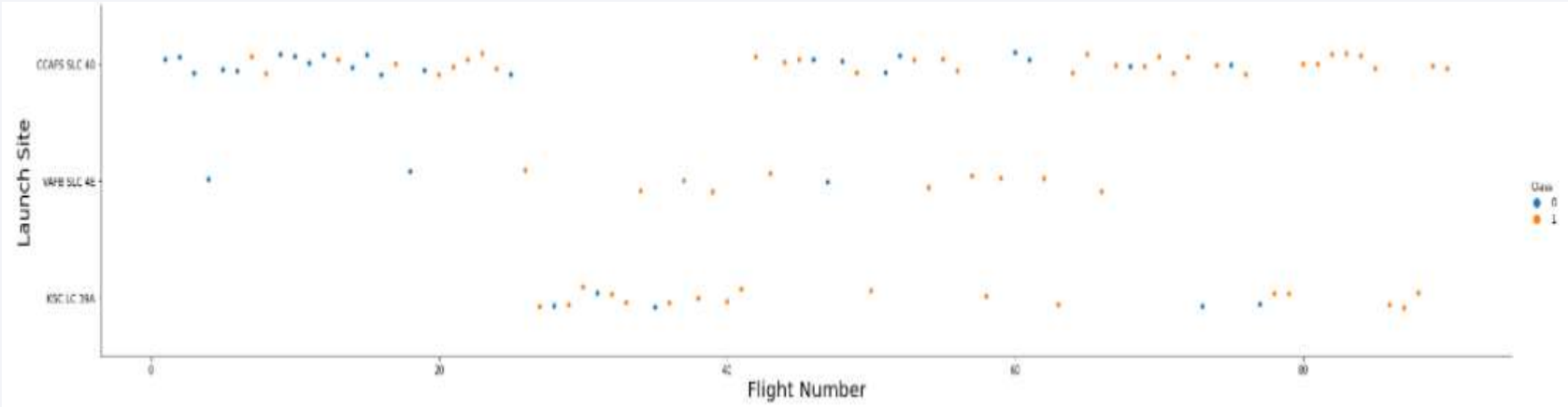
- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Section 2

Insights drawn from EDA

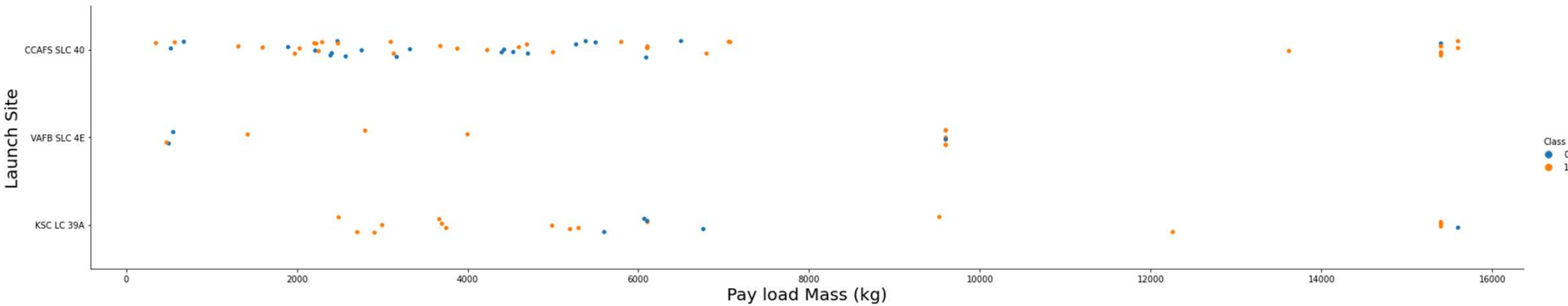
Flight Number vs. Launch Site

- This shows that higher flight numbers were usually met with a higher success at each launch site



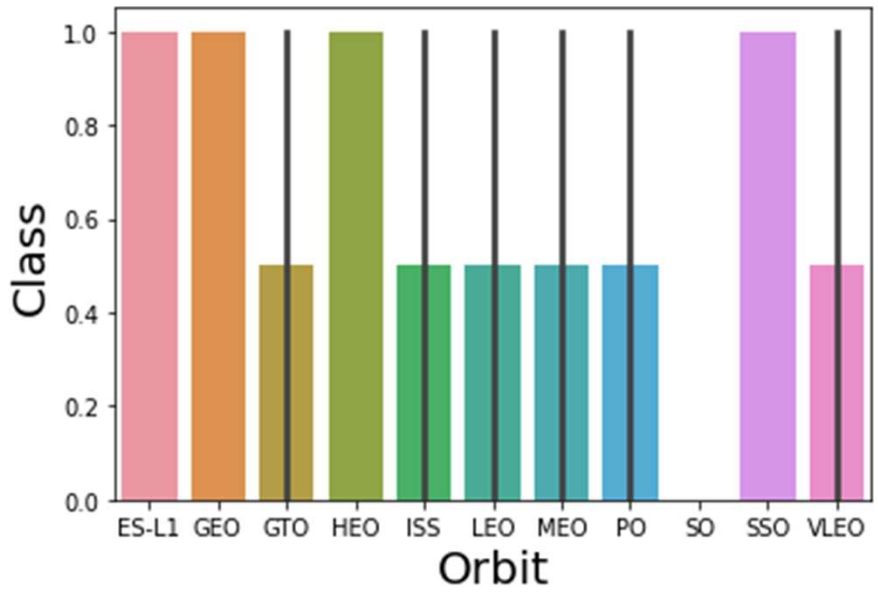
Payload vs Launch Site

- This shows that VAFB SLC never launched a heavy payload



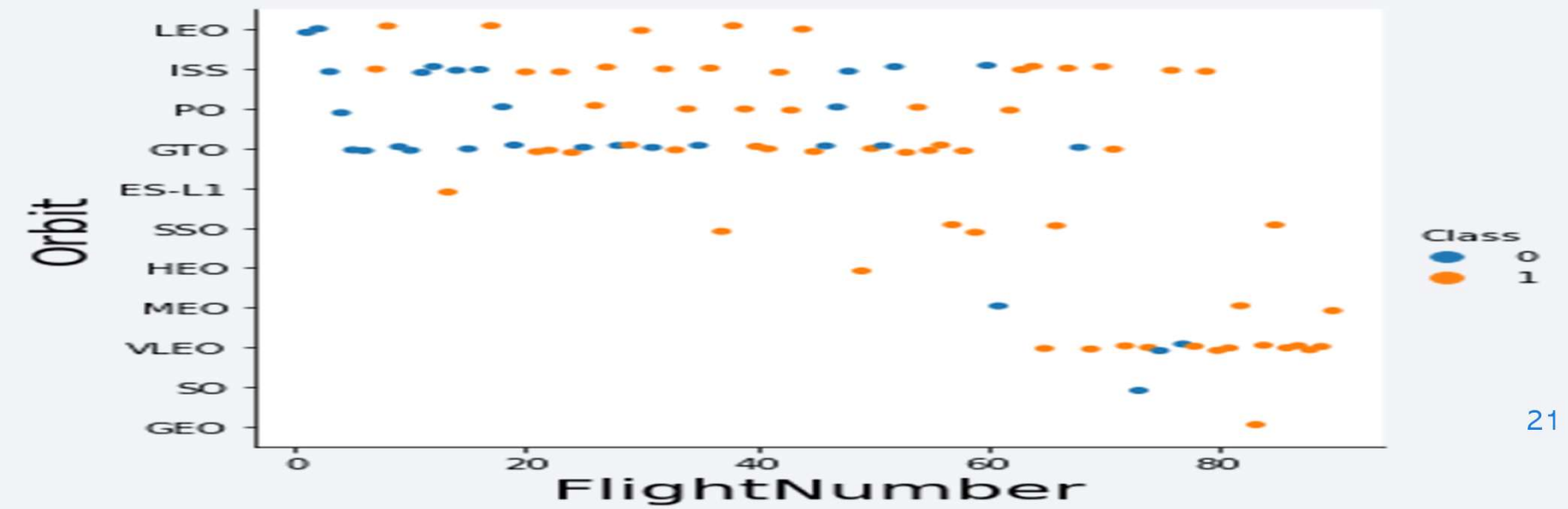
Success Rate vs. Orbit Type

- Certain orbit types had a higher success rate than their counterparts



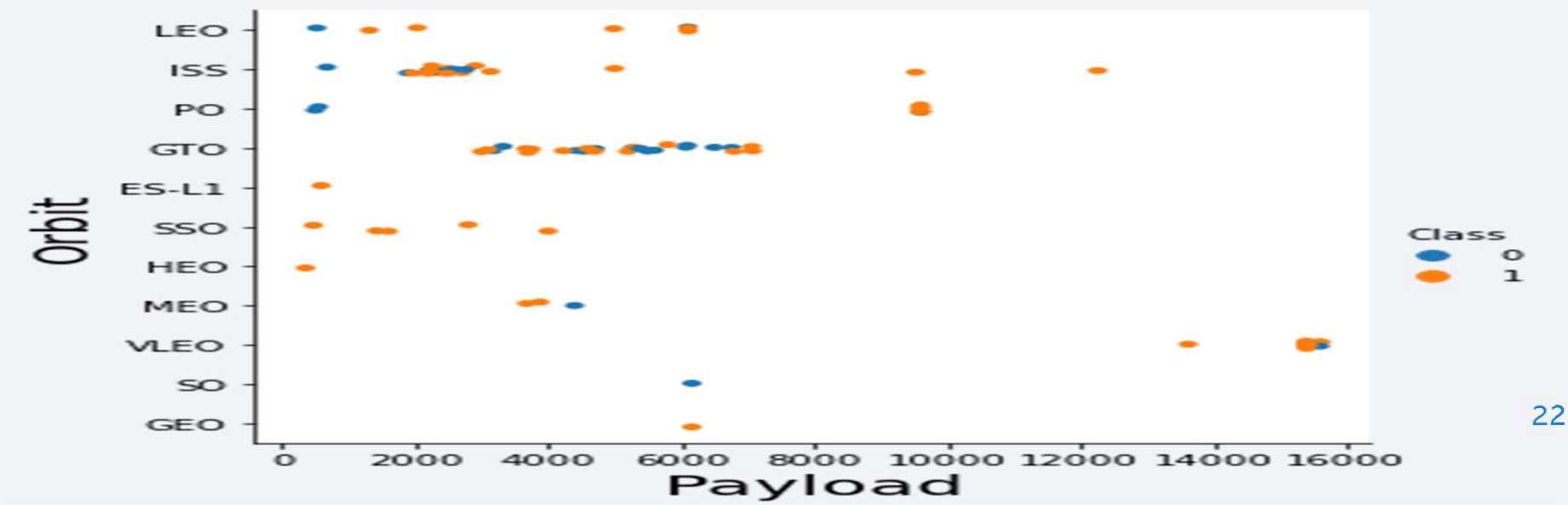
Flight Number vs. Orbit Type

- LEO had a success rate that increased with flight number while GTO did not



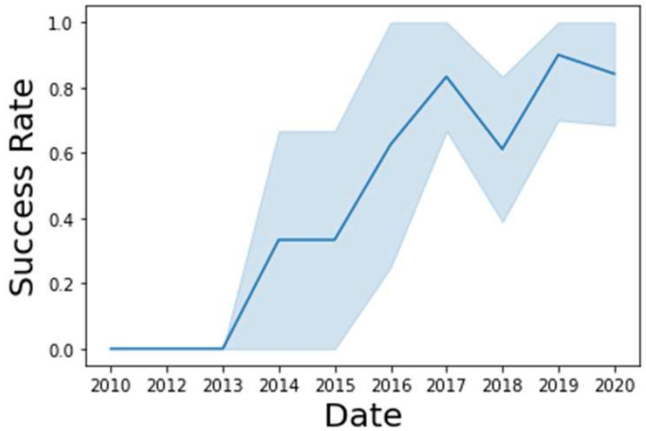
Payload vs. Orbit Type

- PO, LEO, and ISS orbits had higher success rates for larger payloads



Launch Success Yearly Trend

- Starting from 2013 the success rate steadily increased until 2020



All Launch Site Names

- The distinct key word was used to show only unique launch sites from the SpaceX data

Task 1

Display the names of the unique launch sites in the space mission

In [11]:
```
%sql select distinct(LAUNCH_SITE) from SPACEXTBL
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[11]:

launch_site
CCAFS LC-40
CCAFS SLC-40
KSC LC-39A
VAFB SLC-4E

Launch Site Names Begin with 'CCA'

Display 5 records where launch sites begin with the string 'CCA'

In [12]:
```
%sql select * from SPACEXTBL where LAUNCH_SITE like 'CCA%' limit 5
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[12]:

DATE	time__utc_	booster_version	launch_site	payload	payload_mass__kg_	orbit	customer	mission_outcome	landing__outcom
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute
2012-05-22	07:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	00:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- This query was used to find 5 launches from CCA

Total Payload Mass

- We used a sum query to find a total payload mass of 45596 kg

Display the total payload mass carried by boosters launched by NASA (CRS)

In [13]:
```
%sql select sum(PAYLOAD_MASS__KG_) from SPACEXTBL where CUSTOMER = 'NASA (CRS)'
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[13]:

1
45596

Average Payload Mass by F9 v1.1

- The average payload was found using an avg query to be 2928 kg

Task 4

Display average payload mass carried by booster version F9 v1.1

In [14]:
```
%sql select avg(PAYLOAD_MASS__KG_) from SPACEXTBL where BOOSTER_VERSION = 'F9 v1.1'
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[14]:

1
2928

First Successful Ground Landing Date

- A min query was used to find the first successful ground landing

Task 5

List the date when the first successful landing outcome in ground pad was acheived.

Hint:Use min function

In [15]:
```
%sql select min(DATE) from SPACEXTBL where Landing__Outcome = 'Success (ground pad)'
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[15]:

1
2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

- This query was used to find the boosters used in successful drone ship landings with a certain payload mass

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

In [16]:
```
%sql select BOOSTER_VERSION from SPACEXTBL where Landing__Outcome = 'Success (drone ship)' and PAYLOAD_MASS__KG_ > 4000 and
PAYLOAD_MASS__KG_ < 6000
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[16]:

booster_version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

- A count query was used to find the total number of successful and failure outcomes

List the total number of successful and failure mission outcomes

In [17]:
```
%sql select count(MISSION_OUTCOME) from SPACEXTBL where MISSION_OUTCOME = 'Success' or MISSION_OUTCOME = 'Failure (in fligh
t)'
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[17]:

1
100

Boosters Carried Maximum Payload

- A subquery was used to find which boosters carried a maximum payload

Task 8

List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

In [20]:
```
%sql select BOOSTER_VERSION from SPACEXTBL where PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTBL)
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[20]:

booster_version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

- A query was used to find the list of failed landings on a drone ship in 2015

List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

In [21]:
```
%sql SELECT BOOSTER_VERSION, LAUNCH_SITE FROM SPACEXTBL WHERE LANDING__OUTCOME = 'Failure (drone ship)' AND DATE_PART('YEAR
', DATE) = 2015;
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[21]:

booster_version	launch_site
F9 v1.1 B1012	CCAFS LC-40
F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- A list of landing outcomes between 2010-06-04 and 2017-03-20 was found and ranked

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order ¶

In [22]:
```
%sql SELECT LANDING__OUTCOME, COUNT(*) AS QTY FROM SPACEXTBL WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY LAND
ING__OUTCOME ORDER BY QTY DESC;
```

```
 * ibm_db_sa://ygf49439:***@54a2f15b-5c0f-46df-8954-7e38e612c2bd.c1ogj3sd0tgtu0lqde00.databases.appdomain.cloud:32733/bludb
Done.
```

Out[22]:

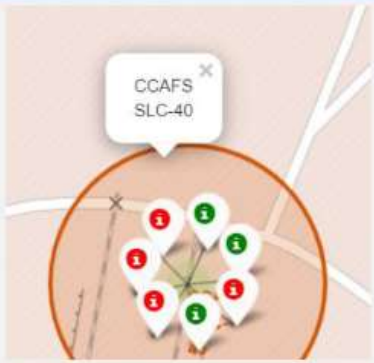
landing__outcome	qty
No attempt	10
Failure (drone ship)	5
Success (drone ship)	5
Controlled (ocean)	3
Success (ground pad)	3
Failure (parachute)	2
Uncontrolled (ocean)	2
Precluded (drone ship)	1

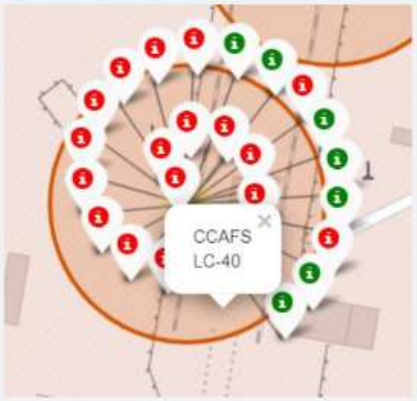
Section 4

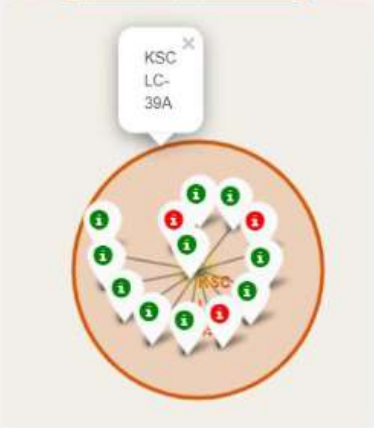
Launch Sites Proximities Analysis

All launch sites global map markers

Markers showing launch sites with color labels







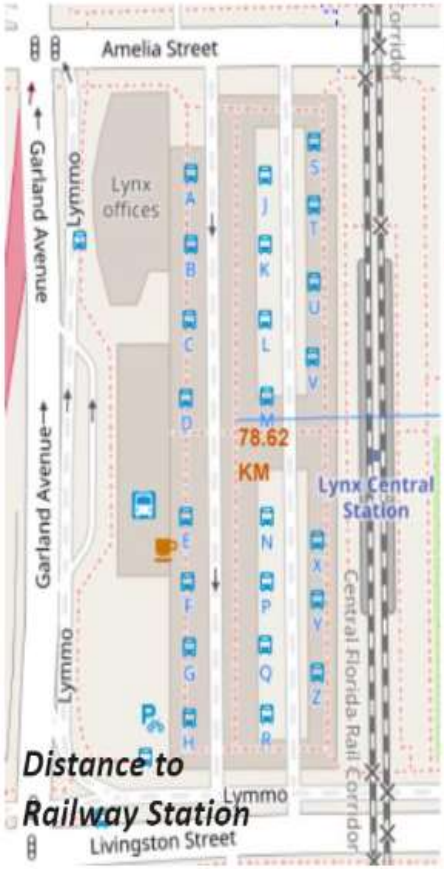
Florida Launch Sites

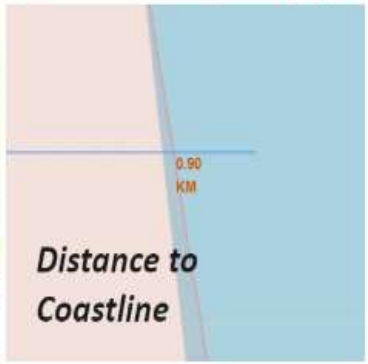
Green Marker shows successful Launches and Red Marker shows Failures

California Launch Site

37

Launch Site distance to landmarks



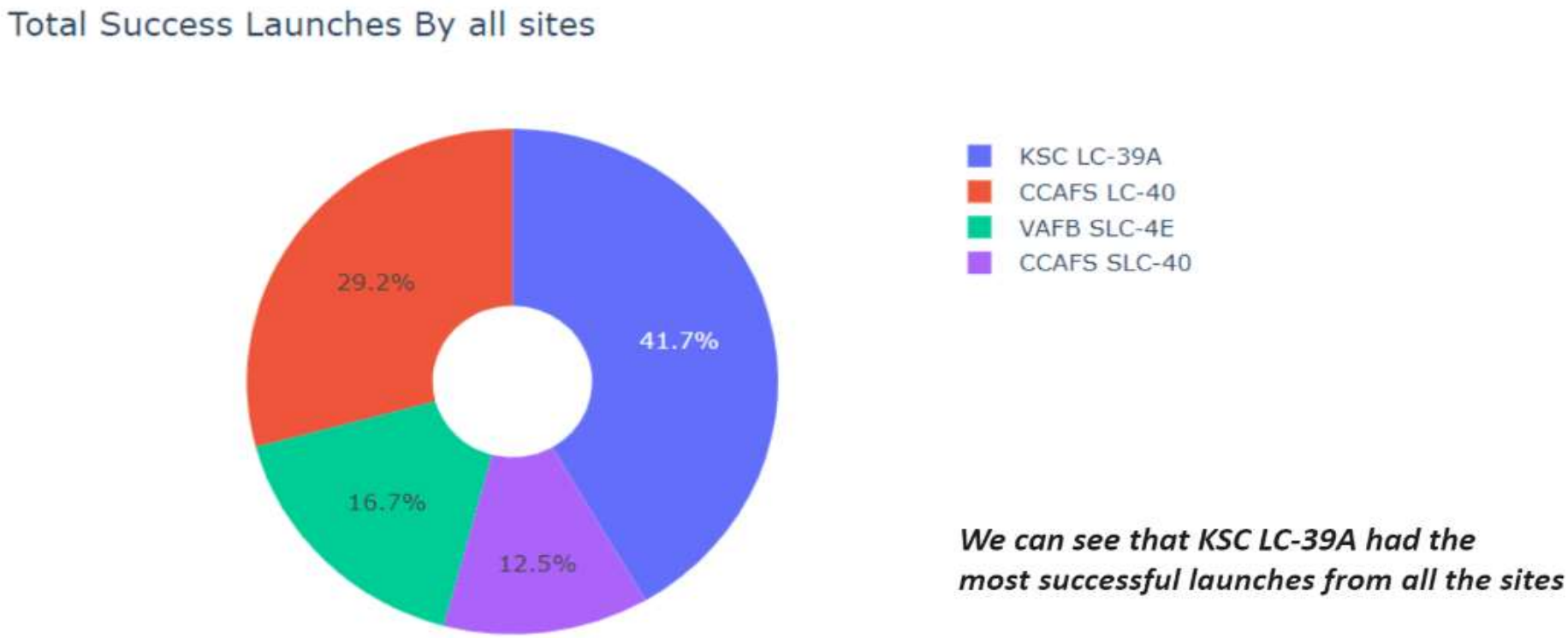


- Are launch sites in close proximity to railways? No
- Are launch sites in close proximity to highways? No
- Are launch sites in close proximity to coastline? Yes
- Do launch sites keep certain distance away from cities? Yes

Section 5

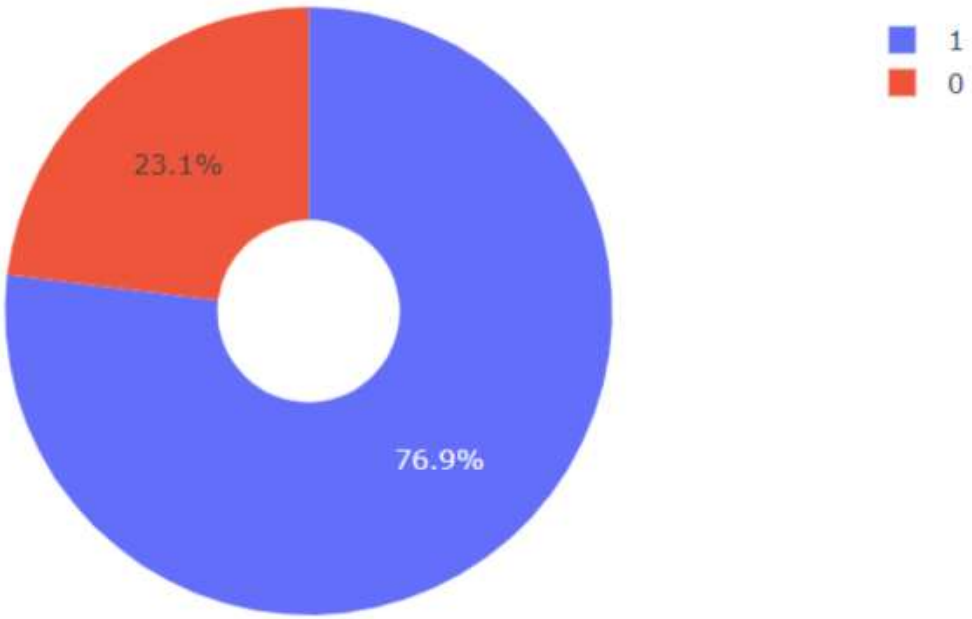
Build a Dashboard with Plotly Dash

Pie chart showing the success percentage achieved by each launch site



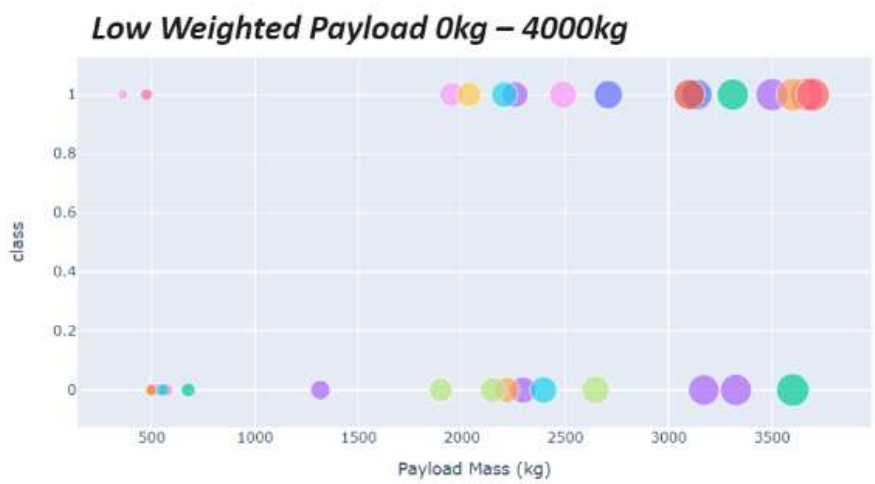
We can see that KSC LC-39A had the most successful launches from all the sites

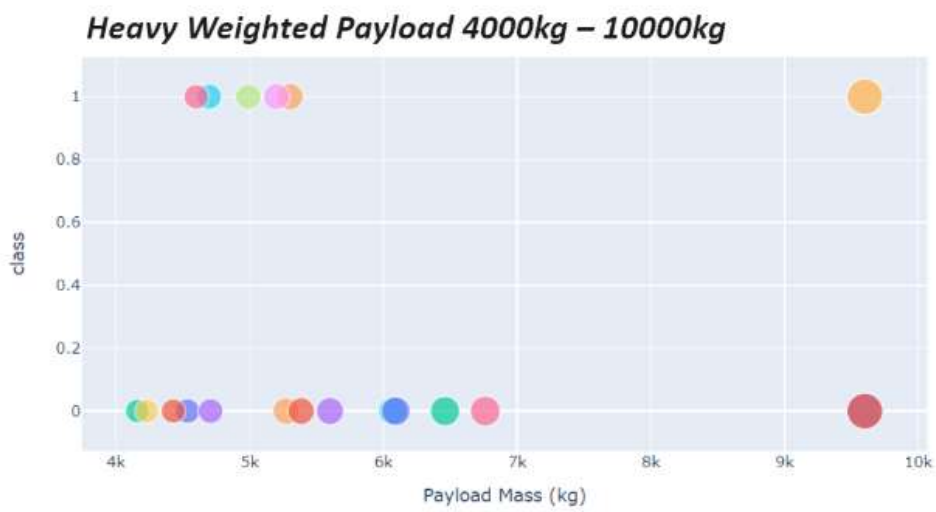
Pie chart showing the Launch site with the highest launch success ratio



KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate

Scatter plot of Payload vs Launch Outcome for all sites, with different payload selected in the range slider





We can see the success rates for low weighted payloads is higher than the heavy weighted payloads

Section 6

Predictive Analysis (Classification)

Classification Accuracy

- The decision tree classifier is the model with the highest classification accuracy

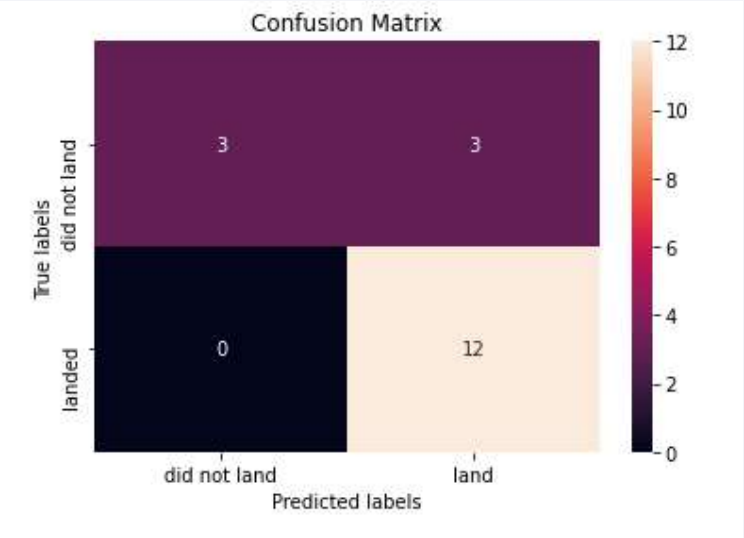
```
models = {'KNeighbors':knn_cv.best_score_,
              'DecisionTree':tree_cv.best_score_,
              'LogisticRegression':logreg_cv.best_score_,
              'SupportVector': svm_cv.best_score_}

bestalgorithm = max(models, key=models.get)
print('Best model is', bestalgorithm,'with a score of', models[bestalgorithm])
if bestalgorithm == 'DecisionTree':
    print('Best params is :', tree_cv.best_params_)
if bestalgorithm == 'KNeighbors':
    print('Best params is :', knn_cv.best_params_)
if bestalgorithm == 'LogisticRegression':
    print('Best params is :', logreg_cv.best_params_)
if bestalgorithm == 'SupportVector':
    print('Best params is :', svm_cv.best_params_)
```

```
Best model is DecisionTree with a score of 0.8732142857142856
Best params is : {'criterion': 'gini', 'max_depth': 6, 'max_features': 'auto', 'min_samples_leaf': 2, 'min_samples_split': 5, 'splitter': 'random'}
```

Confusion Matrix

- The confusion matrix for the decision tree classifier shows that the classifier can distinguish between the different classes. The major problem is the false positives .i.e., unsuccessful landing marked as successful landing by the classifier.



Conclusions

We can conclude that:

- The larger the flight amount at a launch site, the greater the success rate at a launch site.
- Launch success rate started to increase in 2013 till 2020.
- Orbits ES-L1, GEO, HEO, SSO, VLEO had the most success rate.
- KSC LC-39A had the most successful launches of any sites.
- The Decision tree classifier is the best machine learning algorithm for this task.

Thank you!